TalkLate

Traduccion en tiempo real con conciencia cultural y emocional

18. Piramide de pruebas

Trabajo de grado

Julian David Diaz Medina

Diego Alejandro Franco Atehortua

Director: Wider Farid Sanchez Garzon

Version 1.1 - Noviembre de 2026

Version: 1.0 (05/11/2026)

TalkLate se prueba en tres niveles: muchas pruebas unitarias rapidas en la base, menos pruebas de integracion en el medio y pocas pruebas de punta a punta en la punta. Todas corren solas en el CI de GitHub en cada pull request; una falla bloquea el merge.

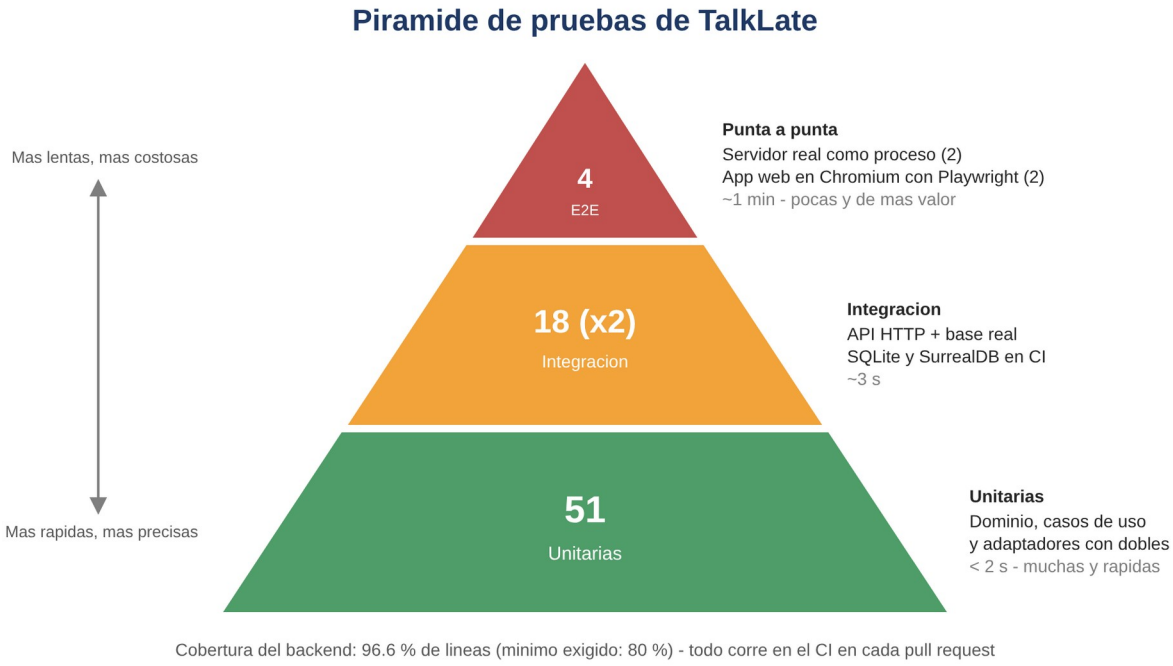


Figura 1. *Piramide de pruebas de TalkLate*

1. Resumen

Nivel	Que prueba	Pruebas	Tiempo	Donde estan	Como se corre
Punta de la piramide: punta a punta (E2E)	La app completa como la usa una persona: servidor real como proceso aparte y app web en un navegador	4 (2 de la API y 2 de la app web)	~1 min	`backend/test/e2e/`, `e2e/pruebas/`	`npm run test:e2e` (backend) y `npm test` (carpeta `e2e/`)
Medio: integracion	La API HTTP completa con una base de datos real (SQLite en memoria y SurrealDB en CI)	29 (18 de la API corren contra SQLite y SurrealDB; 11 de problemas, contra SQLite)	~3 s	`backend/test/integracion/`	`npm run test:integracion` y `npm run test:surreal`
Base: unitarias	Reglas del dominio, casos de uso y adaptadores, con dobles de prueba (sin red ni base)	78 (73 del backend y 5 del frontend)	< 2 s	`backend/test/unitarias/`, `frontend/test/`	`npm run test:unitarias` (backend) y `npm test` (frontend)

Cobertura del backend (unitarias + integracion): **96.5 % de lineas, 94.5 % de funciones y 84.6 % de ramas**. El minimo exigido es 80 % (convencion del proyecto) y el CI falla si baja.

2. Por que una piramide

- **Las unitarias son la base porque son rapidas y precisas.** Prueban una regla a la vez (por ejemplo, "una persona de 12 anos no puede registrarse") sin servidor ni base de datos. Cuando fallan, dicen exactamente que regla se rompio.
- **Las de integracion prueban que las piezas encajan.** La API recibe HTTP, valida, guarda en la base, cifra y responde con los codigos de la convencion REST. Son mas lentas y por eso hay menos.
- **Las de punta a punta prueban el recorrido real.** Son las mas costosas y fragiles, asi que solo cubren los flujos de mas valor: registrarse, conversar por texto y por voz, escuchar la traduccion, reportar y borrar la cuenta.
- La arquitectura hexagonal (ADR-013) es lo que hace barata la base: como el nucleo solo depende de puertos, cada caso de uso se prueba con adaptadores falsos.

3. Base: pruebas unitarias

Archivo	Que cubre	Pruebas
`backend/test/unitarias/dominio.test.js`	Reglas de negocio puras: correo, contrasena, edad minima, nombres, contenido del mensaje; deteccion de idioma, tono y region; prosodia de la voz; expresion de la voz por tono; errores de negocio	14
`backend/test/unitarias/aplicacion.test.js`	Casos de uso con dobles: Circuit Breaker de traduccion (con reloj falso), cadena de voz, vencimiento y limite de audios, paginacion, problemas reportados	12
`backend/test/unitarias/motores.test.js`	Adaptadores de IA con `fetch` falso: Claude (API y Vertex), Qwen, motor de reglas, Whisper, Azure, OpenAI y ElevenLabs; composicion de la cadena de motores	19
`backend/test/unitarias/seguridad.test.js`	Cifrado AES-256-GCM, limites de uso, logs sin datos personales, formato de error, notificador, hash de contrasenas (bcrypt), tokens JWT, validacion de la salida del modelo, limite de uso al reportar problemas y administrador sin contrasena por defecto en produccion	10
`backend/test/unitarias/glosario.test.js`	Glosario cultural: busqueda de modismos paisa y neoyorquinos (sin tildes ni falsos positivos), limite de entradas y que el prompt los incluya	8
`backend/test/unitarias/genero-voz.test.js`	Genero por el pitch de la voz: banda de decision, estimacion de la frecuencia, detector deshabilitado y ffmpeg inexistente	6
`backend/test/unitarias/surreal-cliente.test.js`	Cliente de SurrealDB con `fetch` falso: sesion, reintento con token vencido, literales seguros contra inyeccion	4
`frontend/test/reglas.test.ts`	Reglas de la app: etiqueta de la configuracion, idioma de entrega, resumen del volumen de la voz	5

4. Medio: pruebas de integracion

`backend/test/integracion/api.test.js` levanta la API en memoria (Express + composicion real) y la recorre por HTTP con 18 casos (`problemas.test.js` agrega 11 casos del reporte de problemas): health y OpenAPI, CORS, registro con consentimiento y edad, login y rotacion del token de refresco, recuperacion de contrasena, catalogo, creacion de conversaciones con sus reglas, mensajes por texto y por voz (con transcriptor y sintetizador falsos), historial, aislamiento entre usuarios, reportes, administracion de idiomas, cierre de sesion y borrado de la cuenta.

La misma suite corre dos veces en el CI: contra SQLite (`backend`) y contra un SurrealDB 3.x real (`backend-surreal`). Asi se prueba que los dos adaptadores del puerto `Repositorio` se comportan igual.

5. Punta: pruebas de punta a punta

Prueba	Que hace
`backend/test/e2e/recorrido-usuario.test.js`	Arranca `node src/server.js` como proceso aparte con SQLite en un archivo temporal. Reemplaza Whisper y el motor de voz (API compatible con OpenAI) por servidores HTTP falsos que hablan su mismo protocolo. Recorre registro, conversacion, mensaje escrito, mensaje hablado, voz natural con vector de emocion, historial, reporte y borrado de la cuenta. Tambien comprueba que sin sesion no se puede usar la API.
`e2e/pruebas/traducir.spec.js` (Playwright)	Abre la app web exportada en Chromium con vista de celular (Pixel 7). Inicia sesion, crea una conversacion a Ingles / New York, ve la burbuja de voz, pasa al chat, escribe "Hay que ponerse las pilas con el trabajo" y comprueba la traduccion adaptada "We need to get cracking on the work". Tambien comprueba que una contrasena incorrecta no deja entrar.

Playwright levanta solo el backend y un servidor estatico para la web (`e2e/playwright.config.js`). Guarda capturas y trazas de las pruebas que fallan, y el CI sube el reporte como artefacto.

6. Como correr todo en local

Para levantar la app completa (backend + Expo, y con `--full` tambien Whisper) basta `./run-talklate.sh` en la raiz del repositorio. Para las pruebas:

```
# Backend: base y medio (rapidas)
cd backend
npm run test:unitarias
npm run test:integracion
npm run coverage          # unitarias + integracion con cobertura minima de 80 %
npm run test:e2e          # punta: servidor real como proceso
npm run test:todo         # los tres niveles juntos
npm run arch:check        # reglas de dependencia de la arquitectura hexagonal

# Frontend: unitarias y tipos
cd ../frontend
```

```
npm test
npm run typecheck

# Punta: app web en el navegador
npx expo export --platform web
cd ../e2e
npm install
npx playwright install chromium
npm test
```

7. En el CI (`.github/workflows/ci.yml`)

Job	Pasos
`backend`	unitarias, integracion con cobertura, punta a punta de la API, reglas de dependencia (`arch:check`)
`backend-surreal`	integracion contra SurrealDB 3.x real y verificacion del esquema
`frontend`	unitarias y chequeo de tipos de TypeScript
`e2e-web`	exporta la web y corre Playwright (solo si `backend` y `frontend` pasaron)

8. Reglas del equipo

- Todo cambio de una regla de negocio llega con su prueba unitaria.
- Un endpoint nuevo llega con su caso en la prueba de integracion.
- Las pruebas de punta a punta se reservan para los recorridos de mas valor; no se duplican ahi casos que ya cubre la base.
- Nunca se desactiva una prueba para que el CI pase: se arregla la causa.